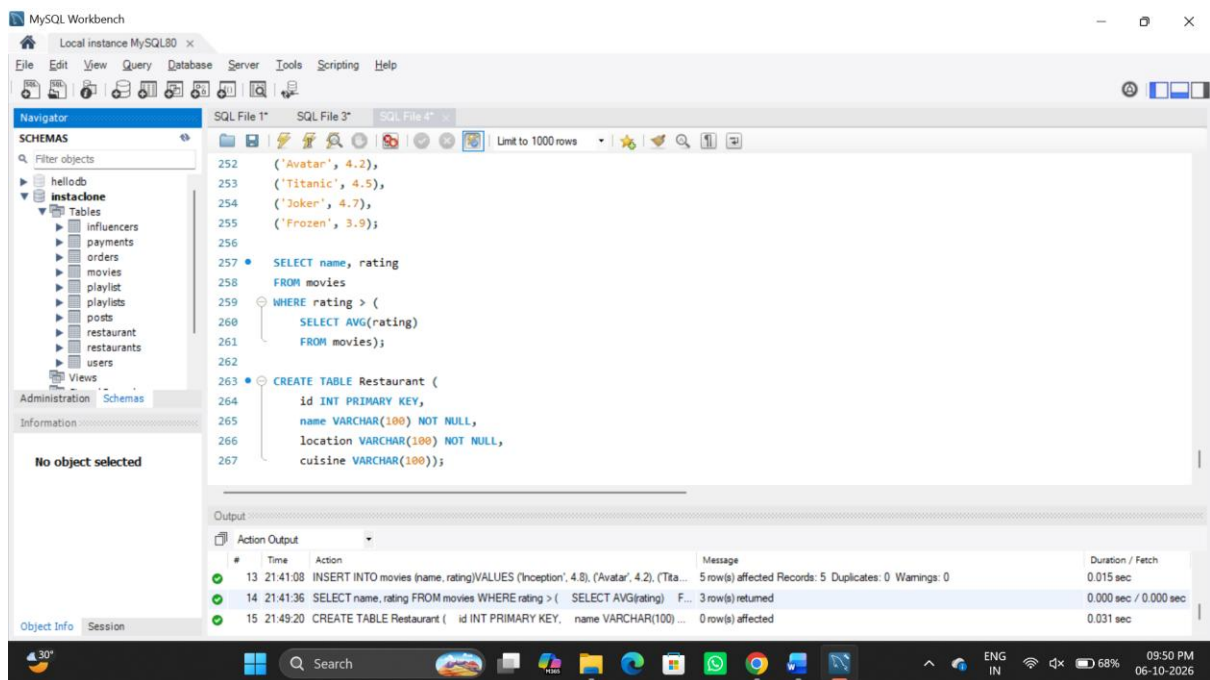
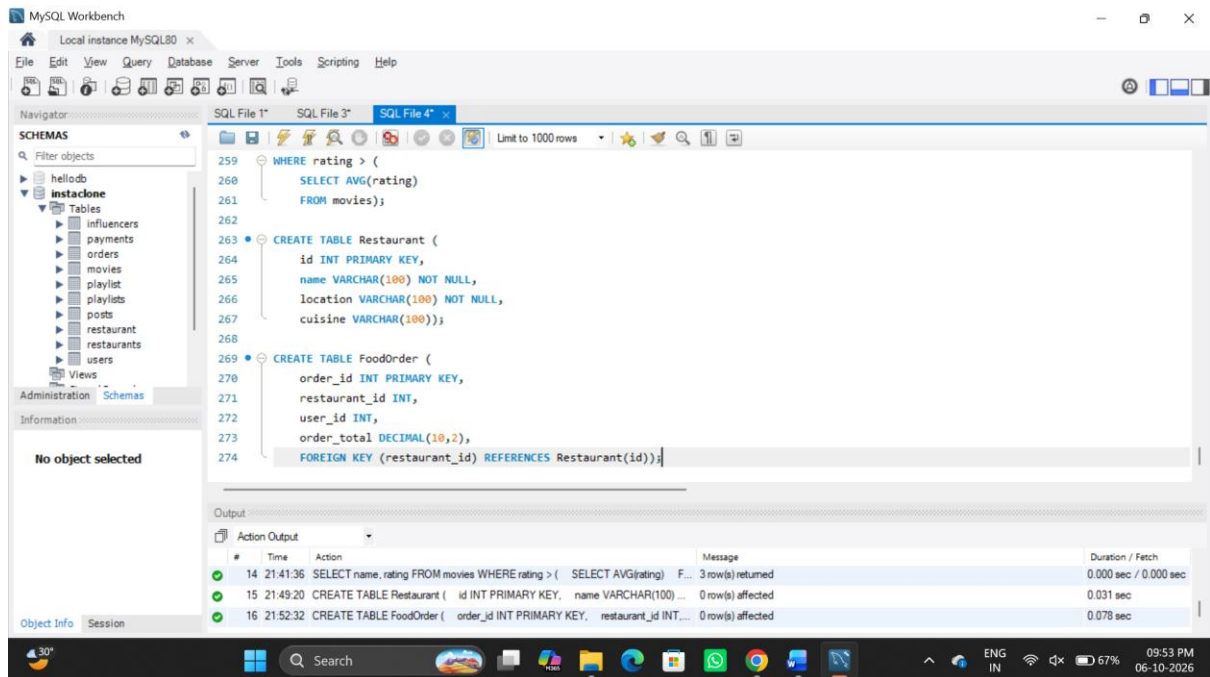


Session – 7

Q1. Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.



Q2. Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).



Q3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following SQL code:

```
283 (1, 'krish_music', 'krish@gmail.com', 'Premium'),
284 (2, 'rahul_beats', 'rahul@gmail.com', 'Free'),
285 (3, 'riya_songs', 'riya@gmail.com', 'Premium'),
286 (4, 'aman_music', 'aman@gmail.com', 'Free'),
287 (5, 'neha_playlist', 'neha@gmail.com', 'Premium'),
288 (6, 'jay_tunes', 'jay@gmail.com', 'Free');
289
290 * SELECT * FROM SpotifyUser;
```

The Result Grid shows the data inserted into the SpotifyUser table:

user_id	username	email	subscription_type
1	krish_music	krish@gmail.com	Premium
2	rahul_beats	rahul@gmail.com	Free
3	riya_songs	riya@gmail.com	Premium
4	aman_music	aman@gmail.com	Free
5	neha_playlist	neha@gmail.com	Premium
6	jay_tunes	jay@gmail.com	Free

The Output tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
17	21:55:15	CREATE TABLE SpotifyUser (user_id INT PRIMARY KEY, username VARCHAR...	0 row(s) affected	0.063 sec
18	21:55:22	INSERT INTO SpotifyUser (user_id, username, email, subscription_type)VALUES (1...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.016 sec
19	21:55:38	SELECT * FROM SpotifyUser LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

Q4. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following SQL code:

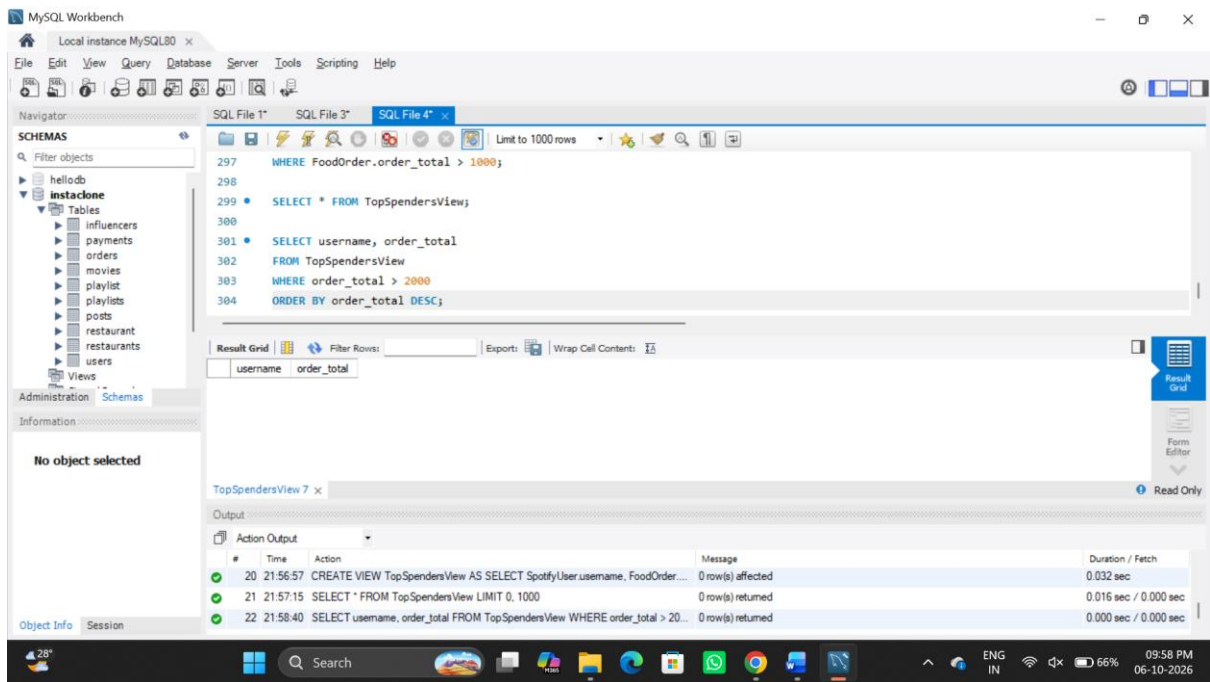
```
292 CREATE VIEW TopSpendersView AS
293 SELECT SpotifyUser.username, FoodOrder.order_total
294 FROM SpotifyUser
295 INNER JOIN FoodOrder
296 ON SpotifyUser.user_id = FoodOrder.user_id
297 WHERE FoodOrder.order_total > 1000;
298
299 SELECT * FROM TopSpendersView;
```

The Navigator panel on the left shows the database schema with tables like influencers, payments, orders, movies, playlist, posts, restaurant, restaurants, and users. The Output panel at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
19	21:55:38	SELECT * FROM SpotifyUser LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
20	21:56:57	CREATE VIEW TopSpendersView AS SELECT SpotifyUser.username, FoodOrder...	0 row(s) affected	0.032 sec
21	21:57:15	SELECT * FROM TopSpendersView LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec

Q5. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.

Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.



The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
297 WHERE FoodOrder.order_total > 1000;
298
299 SELECT * FROM TopSpendersView;
300
301 SELECT username, order_total
302 FROM TopSpendersView
303 WHERE order_total > 2000
304 ORDER BY order_total DESC;
```

The Results tab shows the output of the query, displaying columns 'username' and 'order_total'. The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
20	21:56:57	CREATE VIEW TopSpendersView AS SELECT SpotifyUser.username, FoodOrder...	0 row(s) affected	0.032 sec
21	21:57:15	SELECT * FROM TopSpendersView LIMIT 0, 1000	0 row(s) returned	0.016 sec / 0.000 sec
22	21:58:40	SELECT username, order_total FROM TopSpendersView WHERE order_total > 20...	0 row(s) returned	0.000 sec / 0.000 sec